

Raport de implementare

Adaptive Workplace Dynamics Simulator

Nume student: *[de completat]*

17 iunie 2026

Rezumat

Acest document descrie prototipul Adaptive Workplace Dynamics Simulator (AWDS), o aplicație Python cu interfață Streamlit pentru simularea sintetică a dinamicii unui mediu de lucru. Raportul prezintă funcționalitățile implementate, modelul folosit, structura tehnică, dificultățile întâmpinate, limitările și direcțiile posibile de dezvoltare. Rezultatele numerice urmează să fie completate după rularea scenariilor finale.

Cuprins

1	Descrierea prototipului	3
2	Funcționalități implementate	3
3	Modelul de simulare	4
3.1	Variabile dinamice	4
3.2	Factori statici	4
3.3	Logica generală	4
3.4	Turnover	4
3.5	Reproductibilitate	5
4	Implementare tehnică	5
4.1	Structura proiectului	5
4.2	Arhitectura motoarelor de simulare	5
4.3	Alternativa Mesa	6
5	Scenarii și rezultate	6
5.1	Configurația experimentală	6
5.2	Tabel comparativ	6
5.3	Grafice	6
5.4	Interpretare	7
6	Dificultăți întâmpinate	7
7	Limitări	7

8	Direcții viitoare	8
9	Demo video	8

1 Descrierea prototipului

AWDS este un prototip software care simulează o organizație formată din agenți de tip angajat. Fiecare agent are o stare internă dinamică, care include stres, burnout, productivitate, stare emoțională, energie, satisfacție profesională, echilibru muncă-viață și engagement.

Aplicația este construită ca instrument exploratoriu. Scopul ei nu este să demonstreze relații organizaționale reale, ci să permită observarea unor dinamici sintetice generate de un model configurabil. Astfel, rezultatele trebuie interpretate ca ieșiri ale modelului, nu ca date empirice.

Interfața permite configurarea parametrilor, rularea simulării, vizualizarea graficelor, compararea scenariilor și exportul rezultatelor pentru folosire ulterioară în raport.

2 Funcționalități implementate

Prototipul include următoarele funcționalități principale:

- configurarea numărului de agenți, a numărului de zile simulate și a seed-ului aleator;
- presetări factory pentru scenarii organizaționale predefinite;
- presetări locale definite de utilizator, salvate în `user_presets.json`;
- buton explicit de aplicare a presetării, pentru a evita modificarea accidentală a parametrilor;
- salvarea configurației curente ca presetare nouă;
- controale sticky pentru rulare, pauză/reluare și curățarea rezultatului afișat;
- editarea intervalelor vizibile ale sliderelor prin panoul *Advanced slider ranges*;
- grafice live pentru stres, burnout, productivitate și indicatori secundari;
- selector de motor pentru motorul custom, motorul Mesa sau ambele motoare rulate pe rând;
- mod de comparare a scenariilor folosind același seed și un motor de comparație selectat;
- comparație directă între ieșirile motoarelor atunci când sunt selectate ambele;
- istoric de rulări finalizate în pagina de export;
- căutare în istoricul de export după scenariu, timestamp, seed, zi, număr de agenți sau ID;
- export în formate JSON, CSV, TXT, PNG și ZIP;
- scripturi de pornire pentru Unix/macOS și Windows: `run.sh` și `run.bat`.

3 Modelul de simulare

Fiecare pas al simulării reprezintă o zi de lucru simulată. La fiecare zi, modelul aplică factori organizaționali, factori personali, influență socială, evenimente aleatoare și reguli de actualizare a stării fiecărui agent.

3.1 Variabile dinamice

Agenții au următoarele variabile dinamice principale:

- **stress**: nivelul de stres, normalizat între 0 și 1;
- **burnout**: acumularea burnout-ului, normalizată între 0 și 1;
- **productivity**: productivitatea curentă, normalizată între 0 și 1;
- **emotion**: starea emoțională, între -1 și 1;
- **energy**: energia sau capacitatea de recuperare, între 0 și 1;
- **satisfaction**: satisfacția profesională, între 0 și 1;
- **work_life_balance**: echilibrul muncă-viață, între 0 și 1;
- **engagement**: implicarea agentului în muncă, între 0 și 1.

3.2 Factori statici

Fiecare agent este inițializat cu trăsături personale diferite: rol, reziliență, ambiție, nevoie de suport social, sensibilitate la navetă, încărcătură familială, variabilitate a sănătății, productivitate de bază, sensibilitate la stres și rată de recuperare.

3.3 Logica generală

Modelul folosește ecuații simple și transparente. Stresul crește în funcție de cerințe de lucru, presiune externă și evenimente negative, dar scade în funcție de resurse, recuperare și reziliență. Burnout-ul se acumulează mai lent decât stresul și este influențat de stres ridicat pe termen mai lung. Productivitatea nu este modelată ca o relație liniară de tipul “stresul este mereu rău”. Există un efect de presiune optimă: stresul moderat poate susține productivitatea, în timp ce stresul ridicat și burnout-ul o reduc.

3.4 Turnover

În aplicație, *turnover* înseamnă numărul cumulat de angajați simulați care părăsesc organizația. Plecarea unui agent poate apărea atunci când nivelul de burnout sau stres depășește pragurile configurate. Dacă înlocuirea este activă, un nou agent intră după o întârziere configurată și are temporar o penalizare de productivitate din cauza onboarding-ului.

3.5 Reproductibilitate

Motorul custom folosește `numpy.random.default_rng(seed)`, iar motorul Mesa primește același seed configurat prin modelul Mesa. Același seed, aceeași configurație și același motor selectat duc la aceeași serie de rezultate pentru acel motor. Același seed nu este așteptat să producă traiectorii identice între motorul custom și motorul Mesa, deoarece implementarea și ordinea de activare sunt diferite. Această proprietate este utilă pentru compararea scenariilor în condiții controlate și pentru compararea stabilității tendințelor observate între implementări.

4 Implementare tehnică

Aplicația este implementată în Python. Interfața web este realizată cu Streamlit, iar calculele numerice sunt realizate cu NumPy. Datele sunt organizate și exportate folosind Pandas, iar graficele exportabile sunt generate cu Matplotlib. Pentru grafice interactive în dashboard se folosește Plotly, dacă este disponibil. Implementarea Mesa folosește clasele de model și agent din Mesa împreună cu un spațiu de rețea bazat pe NetworkX.

4.1 Structura proiectului

```
app.py
run.sh
run.bat
simulation/
    config.py
    agent.py
    engine.py
    mesa_engine.py
    engines.py
    scenarios.py
    metrics.py
    exports.py
    utils.py
exports/
requirements.txt
```

4.2 Arhitectura motoarelor de simulare

Prototipul separă acum logica de simulare în spatele unei interfețe de motor. Un motor expune aceleași operații de bază: avansarea cu o zi, rularea până la final, întoarcerea unui snapshot live și întoarcerea rezultatului final. Dashboard-ul folosește `engines.py` pentru a mapa numele motorului selectat la implementarea care trebuie rulată. Astfel, interfața nu depinde de implementarea exactă a motorului.

Motorul custom, implementat în `engine.py`, este o implementare locală, ușoară. El gestionează direct lista de agenți angajați, rețeaua socială, evenimentele aleatoare, actualizările zilnice ale stării, turnover-ul, înlocuirea angajaților și metricele agregate. Avantajul principal este transparența: ecuațiile de actualizare și formatul ieșirilor sunt ușor de inspectat și ajustat.

4.3 Alternativa Mesa

Mesa a fost adăugat ca motor alternativ în `mesa_engine.py`. În această versiune, mediul de lucru este reprezentat ca model Mesa, angajații sunt agenți Mesa, iar relațiile sunt stocate într-un network grid bazat pe NetworkX. Motorul Mesa păstrează aceleași câmpuri de rezultat ca motorul custom, astfel încât dashboard-ul, exporturile și tabelele comparative pot trata uniform ambele motoare.

Scopul adăugării Mesa nu este obținerea unor rezultate identice. Scopul este compararea tendințelor scenariilor în două implementări. Dacă motorul custom și motorul Mesa arată direcții similare pentru aceeași presetare, același seed și aceleași controale de rulare, comportamentul modelului devine mai ușor de interpretat. Dacă ieșirile diferă, diferența ajută la identificarea ipotezelor sau detaliilor de implementare care trebuie cercetate mai atent.

5 Scenarii și rezultate

Această secțiune este un schelet care va fi completat după rularea scenariilor finale și exportul graficelor.

Fluxul de comparație este central pentru prototip. Scenariile pot fi comparate cu motorul custom, cu motorul Mesa sau cu ambele motoare. Astfel se obține o imagine mai clară asupra întrebării de cercetare: dacă efectul observat ține de scenariu sau de diferențele introduse de motorul de simulare.

5.1 Configurația experimentală

- Număr agenți: *[de completat]*
- Număr zile simulate: *[de completat]*
- Seed folosit: *[de completat]*
- Motor sau motoare comparate: *[de completat]*
- Scenarii comparate: *[de completat]*
- Parametri modificați față de presetări: *[de completat]*

5.2 Tabel comparativ

Scenariu	Motor	Stres final	Burnout final	Productivitate finală	Turnover	Burnout cumulativ
<i>[Scenariu 1]</i>	<i>[Motor]</i>	–	–	–	–	–
<i>[Scenariu 2]</i>	<i>[Motor]</i>	–	–	–	–	–
<i>[Scenariu 3]</i>	<i>[Motor]</i>	–	–	–	–	–

Tabela 1: Rezultate sintetice comparative obținute din simulările AWDS.

5.3 Grafice

Inserați aici graficele exportate din aplicație.

Placeholder pentru graficul stres–burnout–productivitate.

Figura 1: Evoluția stresului, burnout-ului și productivității pentru scenariul *[de completat]*.

5.4 Interpretare

[De completat după rularea scenariilor finale. Interpretarea trebuie formulată ca dinamică sintetică a modelului, trebuie să compare tendințele custom și Mesa atunci când sunt folosite ambele motoare și nu trebuie prezentată ca rezultat empiric.]

6 Dificultăți întâmpinate

Principala dificultate a fost proiectarea unui model suficient de expresiv încât să nu reducă dinamica organizațională la reguli triviale de tipul “stresul este rău” și “timpul liber este bun”. Pentru ca simularea să fie utilă, modelul trebuia să permită situații mai nuanțate: de exemplu, un nivel moderat de presiune poate crește productivitatea pe termen scurt, dar presiunea ridicată și susținută poate duce la burnout și scăderea productivității.

O altă dificultate a fost balansarea parametrilor astfel încât scenariile să producă diferențe vizibile, dar fără ca toate rulările să ajungă imediat în extreme. A fost necesară separarea între stres, burnout, energie, satisfacție și productivitate, deoarece aceste variabile nu se mișcă toate cu aceeași viteză și nu răspund identic la aceiași factori.

Configurabilitatea interfeței a reprezentat o provocare practică, deoarece numărul de parametri este mare. Streamlit a simplificat semnificativ partea de UI, permițând construirea rapidă a panourilor de configurare, graficelor și controalelor. Totuși, a fost nevoie de organizarea clară a controalelor, presetări explicite, explicații pentru parametri și un sistem de export ușor de folosit.

Exportul rezultatelor a fost tratat ca parte a experienței de utilizare: utilizatorul trebuie să poată reveni la rulări, să le caute, să identifice fiecare rulare prin ID și timestamp și să descarce datele în formate potrivite pentru raport.

Adăugarea unui al doilea motor a introdus o provocare suplimentară de implementare: ambele motoare trebuiau să producă aceeași schemă de ieșire, astfel încât graficele, exporturile, istoricul de rulări și tabelele comparative să poată reutiliza același cod. Acest lucru a fost important deoarece valoarea de cercetare vine din compararea rezultatelor, nu din menținerea unor fluxuri separate pentru fiecare implementare.

7 Limitări

Cea mai importantă limitare este faptul că modelul folosește date sintetice. Parametrii și ecuațiile sunt configurabile și reproductibile, dar nu sunt calibrate pe date empirice reale. Din acest motiv, rezultatele pot fi folosite pentru a discuta comportamentul intern al modelului, nu pentru a trage concluzii despre companii reale.

Modelul simplifică multe aspecte ale mediului organizațional: relațiile sociale sunt reprezentate printr-o rețea statică, evenimentele sunt modelate probabilistic, iar trăsăturile agenților sunt generate aleator din distribuții simple. Aceste decizii sunt potrivite pentru un prototip, dar limitează realismul simulării.

8 Direcții viitoare

Următorul pas important ar fi obținerea unor date empirice pentru calibrarea modelului. O direcție posibilă este crearea unui formular distribuit persoanelor din companii diferite, prin care să fie colectate informații despre stres perceput, volum de lucru, suport managerial, autonomie, echilibru muncă-viață, satisfacție și intenția de a părăsi organizația.

Scopul unui astfel de formular ar fi să permită transformarea unor experiențe subiective în valori aproximative, adică să se poată “pune un număr pe o senzație”. Aceste valori ar putea fi folosite pentru ajustarea parametrilor modelului, compararea scenariilor sintetice cu răspunsuri reale și identificarea variabilelor care trebuie reprezentate mai bine.

Alte direcții de dezvoltare includ:

- calibrarea parametrilor pe baza răspunsurilor colectate;
- extinderea modelului cu echipe, departamente și roluri organizaționale mai detaliate;
- analiza de sensibilitate pentru a vedea ce parametri influențează cel mai mult rezultatele;
- compararea sistematică între ieșirile motorului custom și ale motorului Mesa;
- salvarea persistentă a rulărilor într-o bază de date;
- îmbunătățirea vizualizărilor și a rapoartelor exportate automat;
- validarea parțială a modelului prin compararea tendințelor sintetice cu date reale anonimizate.

9 Demo video

Demo-ul video va prezenta fluxul principal al aplicației:

1. pornirea aplicației;
2. aplicarea unei presetări factory;
3. ajustarea manuală a unor parametri;
4. salvarea unei presetări locale;
5. selectarea motorului custom, a motorului Mesa sau a ambelor motoare;
6. rularea unei simulări;
7. pauză/reluarea unei rulări live;
8. compararea mai multor scenarii cu motorul de comparație selectat;
9. exportul rezultatelor dintr-un drawer de export;
10. descărcarea fișierelor JSON, CSV, TXT, PNG și ZIP.

Link demo video: *[de completat]*